UNIVERSITÉ LAVAL

Solutionnaire du TP2

À votre tour

0. Préparer les listes

```
res <- read_tsv("/home/public/enrichissement/airway_deseq2.tsv")
```

```
testes   <- res |> filter(!is.na(padj))
de       <- testes |> filter(padj < 0.05, abs(log2FoldChange) > 1)
induits  <- de |> filter(log2FoldChange > 0)
reprimes <- de |> filter(log2FoldChange < 0)
c(univers = nrow(testes), induits = nrow(induits), reprimes = nrow(reprimes))
#>  univers  induits reprimes
#>    18032      531      487
```

L'univers, ce sont les gènes **testés** : ceux qui auraient pu se retrouver dans la liste.

1. L'ORA à la main

```
gc <- AnnotationDbi::select(org.Hs.eg.db, keys = "GO:0051384", keytype = "GOALL",
                            columns = "ENSEMBL")
dans_go <- unique(gc$ENSEMBL)

N <- nrow(testes)                        # l'univers
K <- sum(testes$gene %in% dans_go)       # gènes de l'univers dans le terme
n <- nrow(induits)                       # la liste
k <- sum(induits$gene %in% dans_go)      # gènes de la liste dans le terme
c(N = N, K = K, n = n, k = k, attendu = n * K / N)
#>            N            K            n            k      attendu
#> 18032.000000    92.000000   531.000000    11.000000     2.709184

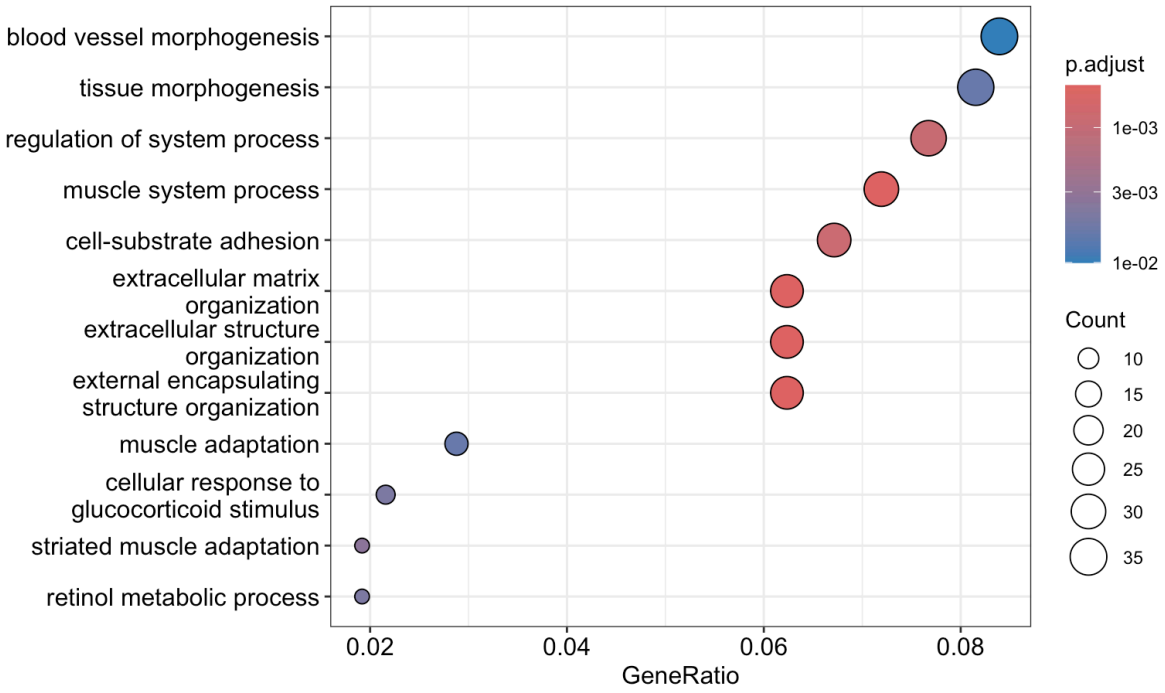
tab <- matrix(c(k, K - k, n - k, N - K - (n - k)), nrow = 2,
              dimnames = list(c("induit", "non induit"), c("dans le terme", "hors du term
tab
#>            dans le terme hors du terme
#> induit                11           520
#> non induit            81         17420
fisher.test(tab, alternative = "greater")$p.value
```

```
#> [1] 8.111353e-05
phyper(k - 1, K, N - K, n, lower.tail = FALSE)
#> [1] 8.111353e-05
```

11 gènes induits dans le terme, pour 2.7 attendus par hasard. `phyper(k - 1, …, lower.tail = FALSE)` donne P(X ≥ k) : la même p-value que le test de Fisher unilatéral. `GOALL` (et non `GO`) inclut les gènes annotés aux **descendants** du terme (règle du chemin vrai).

2. `enrichGO()` sur les induits

```
ora_induits <- enrichGO(gene = induits$gene, universe = testes$gene, OrgDb = org.Hs.eg.db
                        keyType = "ENSEMBL", ont = "BP", readable = TRUE)
nrow(as_tibble(ora_induits))
#> [1] 94
as_tibble(ora_induits) |>
  filter(ID == "GO:0051384") |>
  dplyr::select(Description, GeneRatio, BgRatio, p.adjust, geneID)
#> # A tibble: 1 × 5
#>   Description                GeneRatio BgRatio  p.adjust geneID
#>   <chr>                      <chr>     <chr>       <dbl> <chr>
#> 1 response to glucocorticoid 11/417    92/12376   0.0212 CFLAR/IGFBP2/ERRFI1/FO…
dotplot(ora_induits, showCategory = 12)
```



Le dénominateur de `GeneRatio` est le nombre de gènes induits qui ont **au moins une annotation GO BP** ; celui de `BgRatio`, le nombre de gènes de l'univers annotés. Les gènes sans annotation sont retirés **sans message** : N, n et la p-value diffèrent donc de notre calcul à la main. La `p.adjust` est ajustée pour les milliers de termes testés.

3. Les réprimés

```
ora_reprimes <- enrichGO(gene = reprimes$gene, universe = testes$gene, OrgDb = org.Hs.eg.
                         keyType = "ENSEMBL", ont = "BP", readable = TRUE)
nrow(as_tibble(ora_reprimes))
#> [1] 372
as_tibble(ora_reprimes) |> dplyr::select(Description, GeneRatio, p.adjust) |> head(5)
#> # A tibble: 5 × 3
#>   Description                                          GeneRatio  p.adjust
#>   <chr>                                                <chr>         <dbl>
#> 1 regulation of nervous system development             33/401    0.0000306
#> 2 positive regulation of nervous system development    25/401    0.0000306
#> 3 axon development                                     36/401    0.0000306
#> 4 G protein-coupled receptor signaling pathway         39/401    0.0000306
#> 5 cell morphogenesis involved in neuron differentiation 39/401   0.0000306
```

4. La GSEA

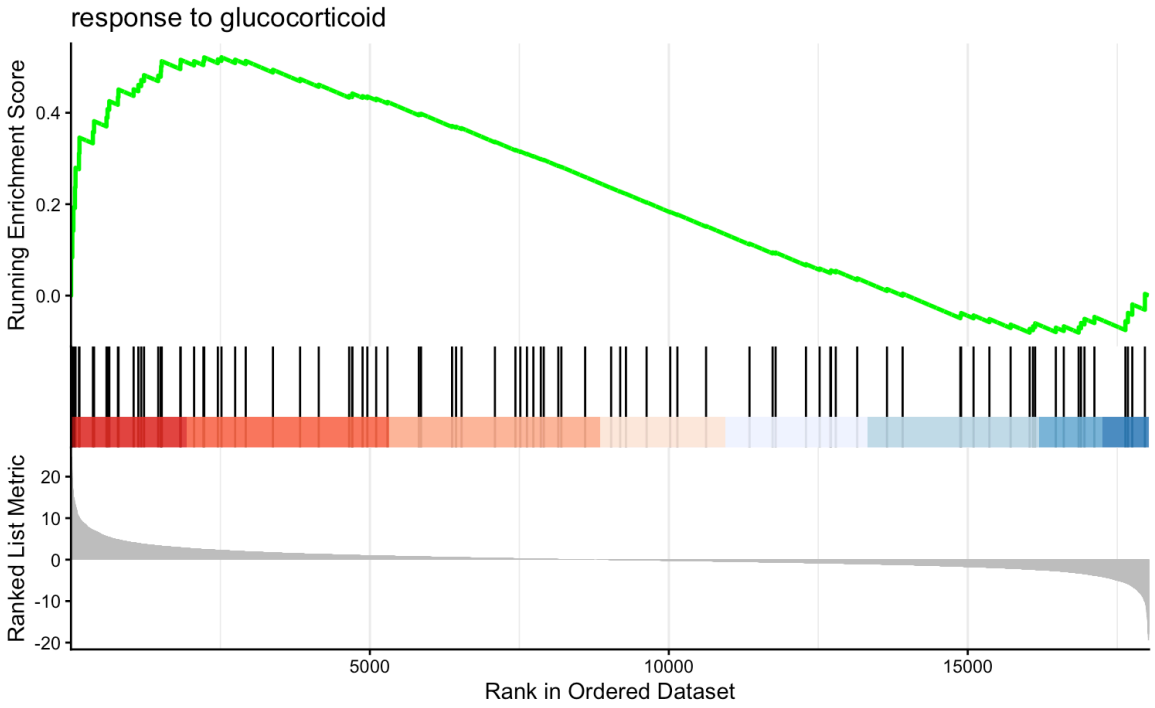
```
classement <- testes |> arrange(desc(stat))
liste_gsea <- setNames(classement$stat, classement$gene)
head(liste_gsea)
#> ENSG00000152583 ENSG00000165995 ENSG00000120129 ENSG00000101347 ENSG00000189221
#>          24.831          24.735          24.187          24.149          23.587
#> ENSG00000211445
#>          22.419

gsea_go <- gseGO(geneList = liste_gsea, OrgDb = org.Hs.eg.db, keyType = "ENSEMBL", ont =
                 minGSSize = 15, maxGSSize = 500, seed = TRUE)
#> Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : The
#> The order of those tied genes will be arbitrary, which may produce unexpected results.
gs <- as_tibble(gsea_go)
nrow(gs)
#> [1] 310
dplyr::count(gs, sens = if_else(NES > 0, "NES > 0", "NES < 0"))
#> # A tibble: 2 × 2
#>   sens        n
#>   <chr>   <int>
#> 1 NES < 0    99
#> 2 NES > 0   211
```

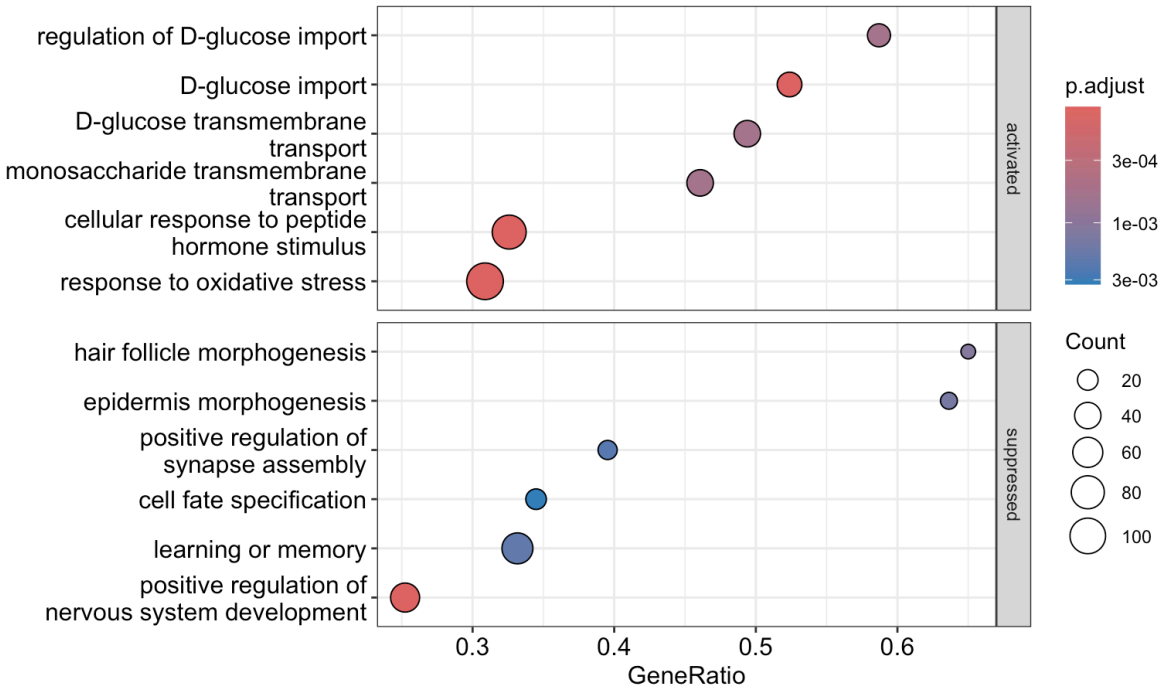
Le classement va du gène le plus **induit** (stat positive et forte) au plus **réprimé**, puisque `res` compare `dex trt vs untrt`. NES > 0 : les gènes de l'ensemble sont en haut du classement, plutôt plus exprimés sous dexaméthasone ; NES < 0 : plutôt moins exprimés. L'avertissement `There are ties in the preranked stats` signale des gènes à égalité, dont l'ordre sera arbitraire.

5. Le graphique de GSEA

```
gs |> filter(ID == "GO:0051384") |> dplyr::select(Description, setSize, NES, p.adjust)
#> # A tibble: 1 × 4
#>   Description                setSize   NES p.adjust
#>   <chr>                        <int> <dbl>    <dbl>
#> 1 response to glucocorticoid      92  1.83  0.00921
gseaplot2(gsea_go, geneSetID = "GO:0051384", title = "response to glucocorticoid")
```




```
dotplot(gsea_go, showCategory = 6, split = ".sign") + facet_grid(.sign ~ ., scales = "fre
```



Les traits (gènes de l'ensemble) se concentrent à **gauche**, en haut du classement : la somme courante monte d'abord, et son pic donne un ES positif. La réponse aux glucocorticoïdes est induite par… un glucocorticoïde : un **contrôle positif** rassurant.

6. Comparer et exporter

```
communs <- intersect(as_tibble(ora_induits)$ID, gs$ID[gs$NES > 0])
length(communs)
#> [1] 42
```

```
write_csv(gs, "gsea_go_airway.csv")
```

Les deux méthodes se recoupent en partie seulement : l'ORA voit les gènes au-delà du seuil, la GSEA des décalages d'ensemble, même modestes.

7. Les Hallmarks

```
hallmarks <- read.gmt("/home/public/enrichissement/h.all.Hs.ensembl.gmt")
```

```
n_distinct(hallmarks$term)
#> [1] 50
nrow(hallmarks)
#> [1] 7331
gsea_h <- GSEA(geneList = liste_gsea, TERM2GENE = hallmarks, seed = TRUE)
#> Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : The
#> The order of those tied genes will be arbitrary, which may produce unexpected results.
gh <- as_tibble(gsea_h)
gh |> dplyr::select(ID, setSize, NES, p.adjust) |> arrange(desc(NES))
#> # A tibble: 20 × 4
#>    ID                                        setSize   NES   p.adjust
#>    <chr>                                       <int> <dbl>      <dbl>
#>  1 HALLMARK_ADIPOGENESIS                         191  1.99 0.00000309
#>  2 HALLMARK_TNFA_SIGNALING_VIA_NFKB              175  1.84 0.0000794
#>  3 HALLMARK_ANDROGEN_RESPONSE                     92  1.74 0.00632
#>  4 HALLMARK_REACTIVE_OXYGEN_SPECIES_PATHWAY       48  1.60 0.0240
#>  5 HALLMARK_OXIDATIVE_PHOSPHORYLATION            198  1.60 0.00632
#>  6 HALLMARK_IL2_STAT5_SIGNALING                  159  1.57 0.00919
#>  7 HALLMARK_UV_RESPONSE_DN                       143  1.57 0.0106
#>  8 HALLMARK_XENOBIOTIC_METABOLISM                161  1.56 0.00804
#>  9 HALLMARK_COMPLEMENT                           148  1.56 0.00936
#> 10 HALLMARK_FATTY_ACID_METABOLISM                139  1.54 0.0141
#> 11 HALLMARK_HYPOXIA                              176  1.48 0.0234
#> 12 HALLMARK_UV_RESPONSE_UP                       140  1.45 0.0309
#> 13 HALLMARK_MITOTIC_SPINDLE                      196  1.43 0.0240
#> 14 HALLMARK_APICAL_JUNCTION                      162  1.38 0.0378
#> 15 HALLMARK_EPITHELIAL_MESENCHYMAL_TRANSITION    197  1.36 0.0453
#> 16 HALLMARK_KRAS_SIGNALING_UP                    149 -1.40 0.0311
```

```
#> 17 HALLMARK_MTORC1_SIGNALING                      194 -1.43 0.0176
#> 18 HALLMARK_E2F_TARGETS                           198 -1.47 0.00984
#> 19 HALLMARK_INTERFERON_ALPHA_RESPONSE              90 -1.54 0.0234
#> 20 HALLMARK_P53_PATHWAY                           184 -1.59 0.00368
```

`read.gmt()` renvoie un tableau à deux colonnes (`term`, `gene`), une ligne par couple ensemble-gène : c'est exactement le format attendu par `TERM2GENE`. 20 Hallmarks sur 50 sont significatifs. C'est un bon point de départ : 50 ensembles **peu redondants**, chacun résumant un processus bien défini, donc une liste courte et lisible, que l'on affine ensuite avec GO ou Reactome. L'adipogenèse arrive en tête : la dexaméthasone fait partie des cocktails utilisés pour différencier les adipocytes en culture.

8. Un ensemble « inflammatoire » induit par un anti-inflammatoire

```
nfkb <- gh |> filter(ID == "HALLMARK_TNFA_SIGNALING_VIA_NFKB")
nfkb |> dplyr::select(setSize, NES, p.adjust)
#> # A tibble: 1 × 3
#>   setSize   NES  p.adjust
#>     <int> <dbl>     <dbl>
#> 1     175  1.84 0.0000794
moteurs <- mapIds(org.Hs.eg.db, keys = str_split_1(nfkb$core_enrichment, "/"),
                  keytype = "ENSEMBL", column = "SYMBOL")
length(moteurs)
#> [1] 58
unname(moteurs)
#>  [1] "DUSP1"    "PER1"     "PTX3"     "DNAJB4"   "TSC22D1"  "CEBPD"
#>  [7] "IRS2"     "KLF9"     "PDLIM5"   "ATF3"     "NR4A3"    "CFLAR"
#> [13] "MAFF"     "MAP3K8"   "GADD45B"  "KLF6"     "SAT1"     "DUSP5"
#> [19] "RHOB"     "TGIF1"    "MYC"      "NFIL3"    "BCL6"     "NAMPT"
#> [25] "GCH1"     "GFPT2"    "STAT5A"   "MCL1"     "CCN1"     "TNIP1"
#> [31] "CCNL1"    "NR4A1"    "INHBA"    "TANK"     "MAP2K3"   "SERPINB8"
#> [37] "TNIP2"    "NFKBIA"   "SGK1"     "TIPARP"   "PFKFB3"   "F3"
#> [43] "TNFAIP3"  "BTG1"     "CSF1"     "EHD1"     "PNRC1"    "GEM"
#> [49] "PANX1"    "PTPRE"    "LDLR"     "ATP2B1"   "ZFP36"    "ZBTB10"
#> [55] "CEBPB"    "RCAN1"    "TNFAIP8"  "FJX1"
```

Parmi les gènes moteurs : *DUSP1*, *NFKBIA* (l'inhibiteur direct de NF-κB), *TNFAIP3* (A20), *ZFP36* (tristétraproline), *TNIP1*, *TNIP2*. Ce sont des **freins** de la voie NF-κB, que les glucocorticoïdes induisent justement pour éteindre l'inflammation. L'ensemble « TNFα via NF-κB » réunit les gènes qui **répondent** à NF-κB, y compris ceux du rétrocontrôle négatif. Un NES positif dit que ces gènes sont plus exprimés, pas que la voie est plus **active**. Il faut toujours lire les gènes moteurs avant de conclure.

Et si… ?

1. Sans univers

```
ora_sans_univers <- enrichGO(gene = induits$gene, OrgDb = org.Hs.eg.db,
                             keyType = "ENSEMBL", ont = "BP")
as_tibble(ora_sans_univers)$BgRatio[1]
#> [1] "419/21338"
c(avec_univers = nrow(as_tibble(ora_induits)), sans_univers = nrow(as_tibble(ora_sans_uni
#> avec_univers sans_univers
#>           94          161
```

Sans `universe`, l'urne contient **tous** les gènes humains annotés, y compris ceux qui ne sont pas exprimés dans ces cellules et ne pouvaient donc pas être DE. Les termes dont les gènes sont simplement exprimés dans ce tissu paraissent enrichis : plus de termes « significatifs », pas plus d'information sur le traitement.

2. Induits et réprimés ensemble

```
ora_tous <- enrichGO(gene = de$gene, universe = testes$gene, OrgDb = org.Hs.eg.db,
                     keyType = "ENSEMBL", ont = "BP")
"GO:0051384" %in% as_tibble(ora_tous)$ID
#> [1] FALSE
```

`GO:0051384` n'est **plus** significatif : ses 11 gènes induits sont dilués dans une liste deux fois plus longue, où les gènes réprimés ne sont presque pas dans le terme. Une ORA mélangée perd des signaux nets dans un sens, et ne dit plus dans quel sens la fonction change.

3. Sans correction

```
ora_brute <- enrichGO(gene = induits$gene, universe = testes$gene, OrgDb = org.Hs.eg.db,
                      keyType = "ENSEMBL", ont = "BP", pAdjustMethod = "none",
                      pvalueCutoff = 0.05, qvalueCutoff = 1)
nrow(as_tibble(ora_brute))
#> [1] 553
n_termes_testes <- nrow(ora_brute@result)
round(n_termes_testes * 0.05)
#> [1] 186
```

553 termes au lieu de 94. Sur les 3714 termes testés, environ 186 passeraient le seuil de 5 % par le seul hasard.

4. Mauvais ordre, mauvais signe

```
gseGO(geneList = sort(liste_gsea), OrgDb = org.Hs.eg.db, keyType = "ENSEMBL", ont = "BP")
#> Error in GSEA_internal(geneList = geneList, exponent = exponent, minGSSize = minGSSize
```

Un tri croissant produit une **erreur** : `gseGO()` vérifie l'ordre. Mais `-stat` (ce qu'on obtient avec `untrt vs trt`) donne une liste tout aussi décroissante, donc **aucune erreur** :

```
liste_inverse <- sort(-liste_gsea, decreasing = TRUE)
gsea_inv <- gseGO(geneList = liste_inverse, OrgDb = org.Hs.eg.db, keyType = "ENSEMBL", or
                  minGSSize = 15, maxGSSize = 500, seed = TRUE)
#> Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : The
#> The order of those tied genes will be arbitrary, which may produce unexpected results.
as_tibble(gsea_inv) |> filter(ID == "GO:0051384") |> dplyr::select(Description, NES, p.ad
#> # A tibble: 1 × 3
#>   Description                 NES p.adjust
#>   <chr>                     <dbl>    <dbl>
#> 1 response to glucocorticoid -1.81  0.00945
```

Le NES change de **signe** : la réponse aux glucocorticoïdes paraîtrait réprimée par un glucocorticoïde. Lire l'en-tête de `res` **avant** l'enrichissement.

5. Classer par log2FoldChange

```
par_lfc <- testes |> arrange(desc(log2FoldChange)) |> head(10)
par_stat <- testes |> arrange(desc(stat)) |> head(10)
par_lfc |> dplyr::select(gene, baseMean, log2FoldChange, stat)
#> # A tibble: 10 × 4
#>    gene            baseMean log2FoldChange  stat
#>    <chr>              <dbl>          <dbl> <dbl>
#>  1 ENSG00000179593    67.2            9.51  9.02
#>  2 ENSG00000109906   385.             7.35 13.7
#>  3 ENSG00000250978    56.3            6.33  9.36
#>  4 ENSG00000132518     5.65           5.88  4.47
#>  5 ENSG00000127954   286.             5.21 10.6
#>  6 ENSG00000249364     8.84           5.09  4.42
#>  7 ENSG00000171819   255.             5.08  6.71
#>  8 ENSG00000100033    55.4            4.89  9.40
#>  9 ENSG00000260802     6.72           4.88  4.09
#> 10 ENSG00000168309   160.             4.84 14.6
length(intersect(par_lfc$gene, par_stat$gene))
#> [1] 0
```

En tête du classement par log2FoldChange, des gènes **peu exprimés** (faible `baseMean`) dont l'effet est grand mais très incertain. `stat` = log2FC / erreur standard combine l'**effet** et la **précision** : un gène très induit mais mal mesuré n'arrive pas en tête.